

# Technical Dossier

## Edge-Deployed Waste Classification with Sensor-Augmented Vision (CAPS-FCL)

BITE497J Project I | Ansh Jerath (23BIT0123), Arnav Mishra (23BIT0142), Bhavesh Singh Thakur (23BIT0199) |  
Guide: Dr. Valarmathi B.

*Written in simple English for the review. Every number in this dossier comes from a saved result in the project folder. Where something is not done or not proven, it says so.*

### How to use this dossier

- Section 1 to 3: the idea in plain words. Read these first.
- Section 4 to 9: data, model, training, results, analysis and the safety parts, each with what, how, why and where the code is.
- Section 10 to 12: extra parts, the hardware prototype, and the honest limits.
- Section 13: a map from every feature to the file that codes it. Section 14: how to run the demo in 10 minutes. Section 15: questions the professor may ask, with answers. Section 16: words explained.

### 1. What the project does (in plain words)

A waste sorting machine looks at one item, decides what it is, and sends it to the right bin. Most machines use one camera and one model, give one answer, and never say how sure they are. This project builds a smarter sorter in software:

- It recognises 33 kinds of items, grouped into 9 material families (plastic, paper, cardboard, glass, metal, organic, styrofoam, textile, hazardous). One family = one bin.
- It says how sure it is. If it is unsure, a person checks the item.
- It never sends a dangerous item (battery, electronic waste, medical waste) to recycling. The operator can set how rare a mistake must be, and the system keeps to it for items like the ones it was calibrated on.
- Two cheap sensors (humidity or moisture, and gas) tell if an item is dirty with food, which a camera cannot see.
- It can explain its decisions with heat maps, and it can learn together with other machines without sharing photos (federated learning).

**Status in one line: the software is built and tested on real held-out images. The sensors are simulated, and a small physical prototype with an ESP32 is planned. This is Technology Readiness Level 3 (proof of concept).**

### 2. The problem and the goals

| Problem in existing sorters          | What we did                                                       |
|--------------------------------------|-------------------------------------------------------------------|
| one model, one answer, no confidence | uncertainty with monte carlo dropout and calibrated probabilities |

|                                                        |                                                                                 |
|--------------------------------------------------------|---------------------------------------------------------------------------------|
| only a few broad classes                               | 33 classes, 9 families, and results reported at item level and family level     |
| a wrong hazard call is treated like any other mistake  | hazard-safe rules: two-tier gate and conformal family sets with a stated limit  |
| camera cannot see food residue                         | organic contamination index from two sensors, with a model for each sensor case |
| sending photos to a server costs bandwidth and privacy | federated averaging: only model weights move (simulated)                        |

### 3. The big picture

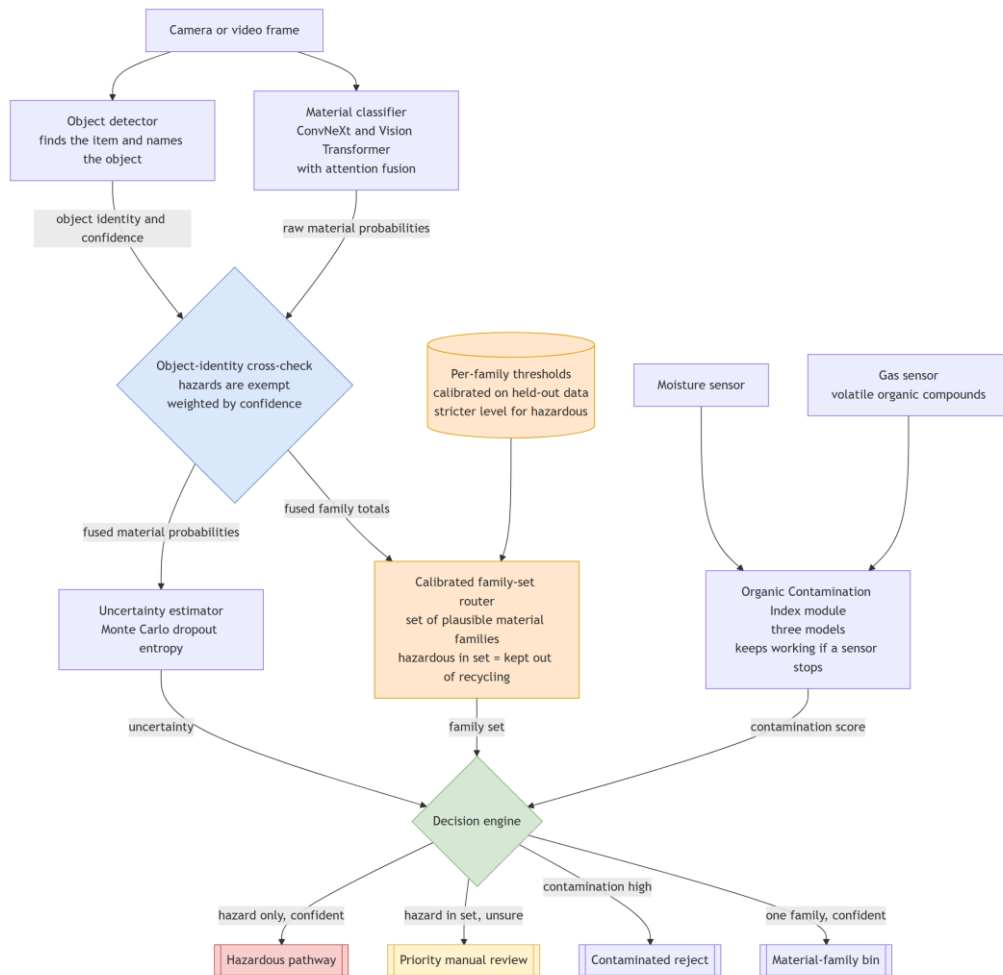


Figure 1. System: camera, two models, cross-check, uncertainty, family sets, sensors, decision engine

#### What happens to one item, step by step:

- 1. The camera takes a picture. The detector (YOLO) finds the item and names the object (for example: can).
- 2. The classifier (ConvNeXt + Vision Transformer) gives a probability for each of the 33 classes.
- 3. The cross-check lowers materials that do not fit the object. Hazardous classes are never lowered.

- 4. Uncertainty is measured (monte carlo dropout). A set of possible families is built (conformal prediction).
- 5. The sensors give a contamination score.
- 6. The decision engine picks one action: hazardous bin, priority review, contamination reject, or the bin of the family.

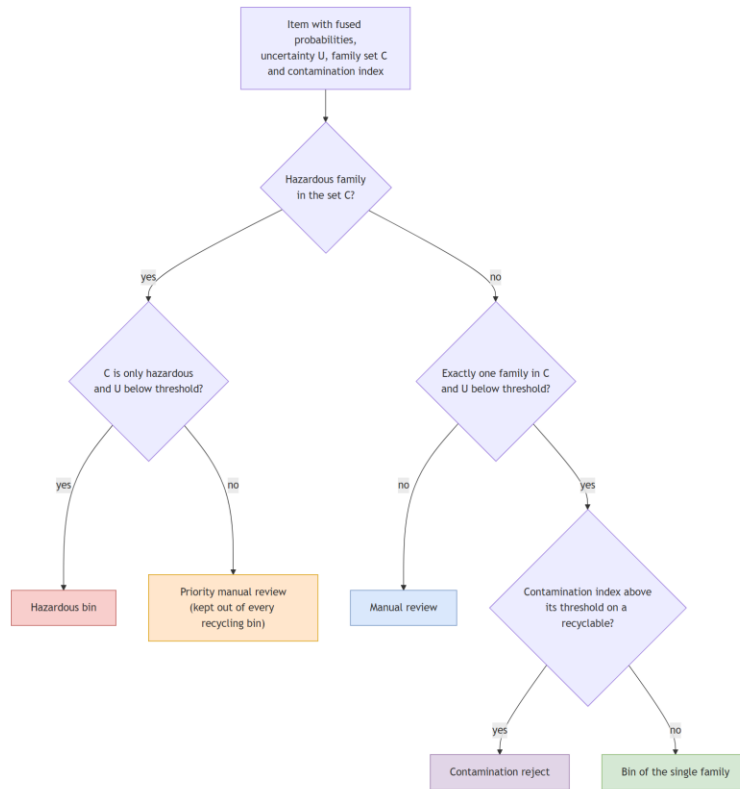


Figure 2. Decision flow

## 4. Data

We use three public Kaggle datasets (the notebook downloads them). The 30-class household dataset is the main part. TrashBox is used only for electronic waste and medical waste, and Garbage Classification 12 gives battery, clothing, shoes and extra food waste. Folders that are too broad (for example one big 'plastic' folder) are dropped, because they cannot be put in one of our fine classes without guessing.

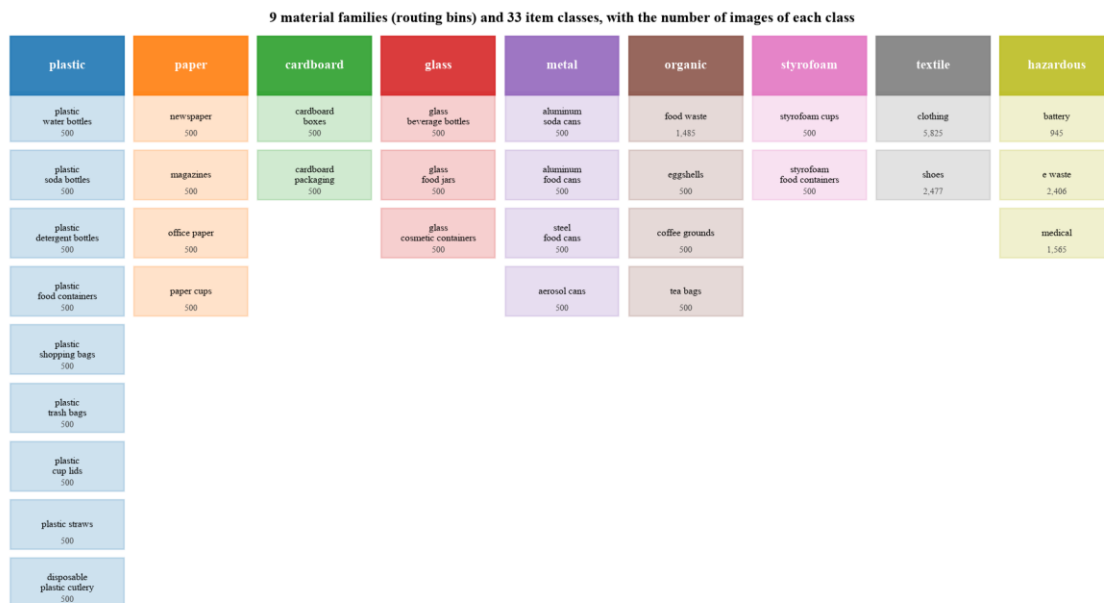


Figure 3. The 33 classes in 9 families, with image counts

| Item                      | Value                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------|
| total images              | 28,203                                                                                  |
| train / validation / test | 19,739 / 4,232 / 4,232 (70 / 15 / 15, stratified, seed 42)                              |
| class imbalance           | largest class is 11.6 times the smallest (clothing 5,825 vs about 500 for most classes) |
| code                      | taxonomy.py (classes and sources), ingest_raw_datasets.py, make_data_splits.py          |

Why stratified: every class keeps the same share in train, validation and test, so rare classes are tested too.

Augmentation (training images only): resize to 256, random crop 224, flip, rotation up to 20 degrees, colour change, blur, random erasing. This copies camera pose, light and partly hidden items. Validation and test are never changed. Code: build\_transforms in data/waste\_image\_dataset.py.

## Two data bugs we found and fixed

- Split leak: the file name hash used the operating system path, so Windows and Colab made different splits. Scoring the Colab model on a split built on Windows gave 94.52%, which was too high because about 70% of the 'test' images had been used for training. We found it because the number went up. Fixed with POSIX paths, and the exact Colab split was recovered (the model scores exactly 3,779 of 4,232 = 89.30% on it).
- Stale duplicates: one Colab run showed 1,234 steps per epoch instead of 617, because old copies of the images were still on the drive. The split now ignores files not listed in the latest provenance file, and the notebook has a gate that stops if sizes differ.

## 5. The model

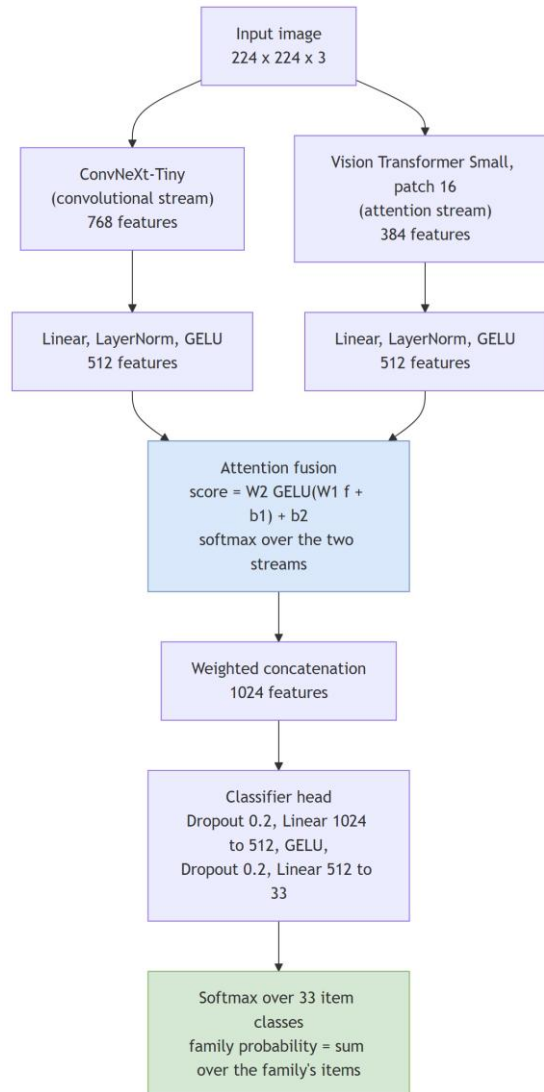


Figure 4. Hybrid classifier

Two pretrained networks look at the same 224 by 224 picture:

- ConvNeXt-Tiny (27.8 million parameters): a convolutional network, good at texture (foil, cardboard fibres).
- Vision Transformer Small (21.7 million parameters): looks at the whole picture with attention, good at shape and context.
- Each gives a feature vector, projected to 512 numbers (linear layer, layer norm, GELU).
- An attention layer gives each stream a weight (they add up to 1), so the model can trust one more for a given image. We measured it: ConvNeXt gets 82% to 93% of the weight for every family.
- The two weighted vectors are joined (1,024 numbers) and a small head (dropout 0.2, 1024 to 512, GELU, dropout 0.2, 512 to 33) gives the 33 scores. Total 50.75 million parameters.

*The attention in words: score = a small network applied to each stream; weight = softmax of the two scores (so weights are positive and add to one).*

### Why these choices:

- Transfer learning: training from scratch gave only 49% in an earlier try (on an earlier dataset), so we start from ImageNet weights.
- GELU: smooth activation used by both pretrained networks; no dead neurons.
- LayerNorm instead of BatchNorm: the result does not depend on batch size, so it behaves the same on the edge with one image.
- Dropout 0.2 in the head: regularisation, and it is also what monte carlo dropout uses later.
- Code: classification/hybrid\_model.py, classes AttentionFusion and HybridConvNeXtViT.

## 6. Training

| Setting                            | Value                                                                         | Why                                                                                                 |
|------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| optimiser                          | AdamW, learning rate 3e-4, weight decay 0.05                                  | adaptive steps suit pretrained networks plus a new head; weight decay is applied correctly in AdamW |
| learning rate schedule             | one-cycle: warm-up for 1 epoch, then slow decay                               | warm-up protects pretrained weights                                                                 |
| loss                               | cross-entropy with label smoothing 0.1                                        | stops over-confidence between look-alike classes                                                    |
| batch size / epochs                | 32 / up to 25 with early stopping (patience 5)                                | stop when validation stops improving                                                                |
| mixed precision, gradient clip 1.0 | on                                                                            | speed on the GPU; stability                                                                         |
| class imbalance                    | balanced sampler only (improved recipe)                                       | the first recipe used sampler and loss weights together, which corrects twice                       |
| backbones                          | frozen for 2 epochs, then 10 times lower learning rate than new layers (3e-5) | protects pretrained features (see below)                                                            |

### The improved recipe and why it was needed

Our first model reached 89.30%. A simple SVM on frozen ImageNet features reached 93.3%, which is better. That told us fine-tuning had damaged the pretrained features: the new head was random, so large noisy gradients flowed into the good backbones from step one. The fix: train the head first with frozen backbones (2 epochs), then fine-tune the backbones slowly, and correct imbalance only once. Result on Colab: 93.83% item accuracy.

### The real Colab run (executed notebook)

- Tesla T4 GPU, 617 steps per epoch, about 4 minutes per epoch.
- Validation accuracy 0.900 after epoch 1 (only the head trained), best 0.939 at epoch 8, training stopped after epoch 13 by early stopping, total about 52 minutes.
- Training accuracy ends at 0.975 and validation near 0.938: a small gap. This is mild overfitting, not harmful.
- Test set: item accuracy 93.83%, family accuracy 97.28%, hazard recall 98.37% (726 of 738).

- The executed notebook is notebooks/colab\_classifier\_training\_executed.ipynb; the annotated copy with comments is review/notebooks/06\_colab\_training\_executed\_annotated.ipynb. Training code: classification/train\_classifier.py, function train.

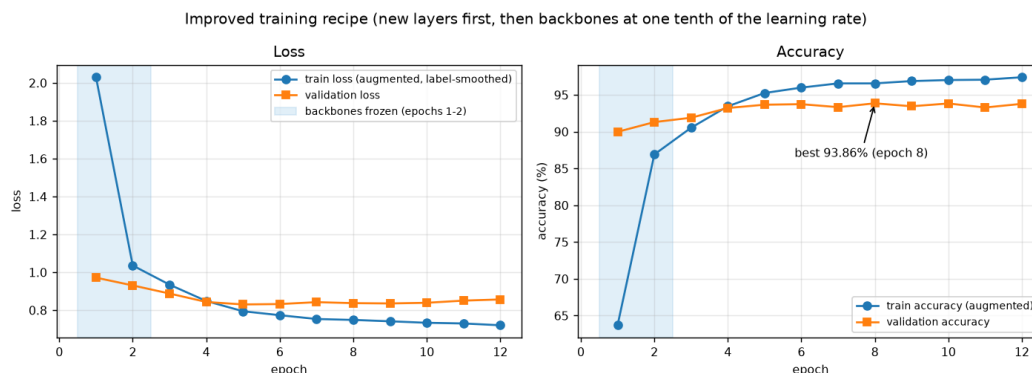


Figure 5. Training history of the improved recipe

## 7. Results

| Metric (4,232 test images) | First model         | Improved recipe (Colab)     |
|----------------------------|---------------------|-----------------------------|
| item accuracy (33 classes) | 89.30%              | 93.83%                      |
| family accuracy (9 bins)   | 94.14%              | 97.28%                      |
| hazard recall              | 93.50% (690 of 738) | 98.37% (726 of 738)         |
| macro F1                   | 0.863               | 0.9065 (from the Colab log) |
| top-3 accuracy             | 96.74%              | not computed                |

The detailed analysis (calibration, imbalance, PCA, LDA, SVM, bootstrap intervals, confusion matrices) was made on the first model, because its test split was recovered and verified. The improved recipe's numbers are from the Colab run itself. A local re-score gave 96.8%, which is too high and is a leak signature (the local split differs from Colab's), so it was discarded. The evaluation command now refuses a split that does not reproduce the validation accuracy stored in the checkpoint.

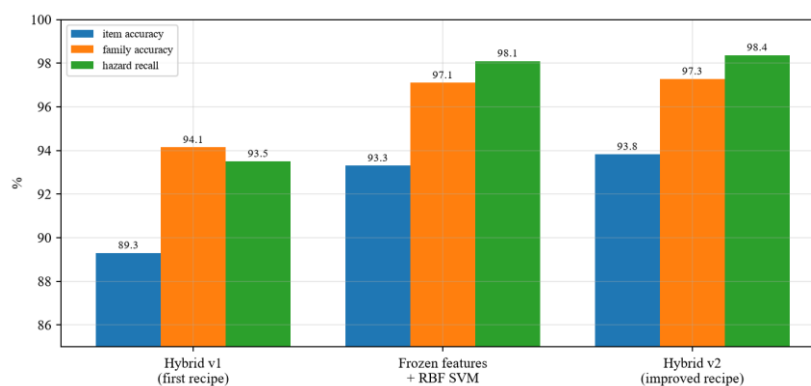


Figure 6. Item accuracy, family accuracy and hazard recall

Why two accuracies? A bin is chosen by family. Mixing two kinds of plastic bottle is a recognition error but not a sorting error. Seven of the eight most common mistakes of the first model are inside one family (cardboard boxes and packaging, steel and aluminium food cans, electronic waste and medical waste).

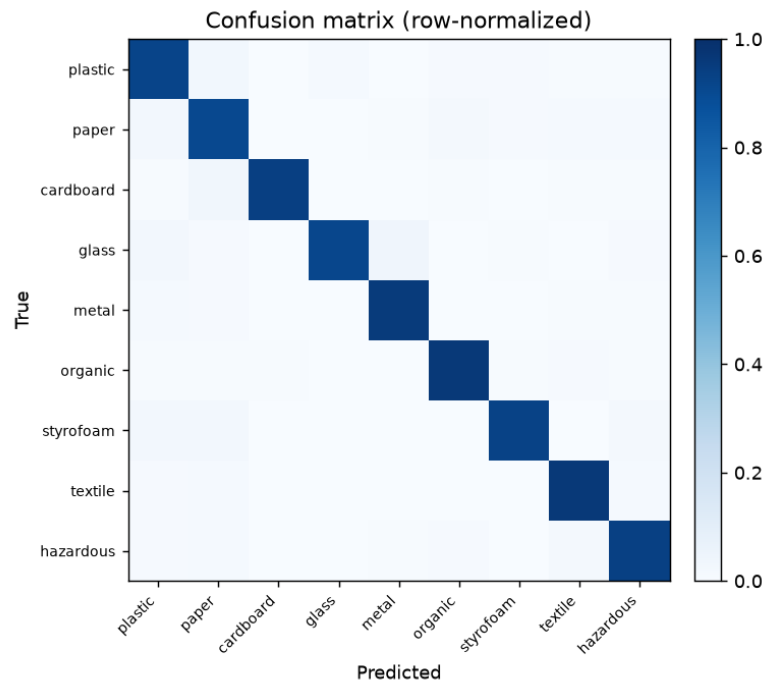


Figure 7. Confusion matrix at family level (first model)

## Detector

| Variant            | Precision | Recall | mAP50 | mAP50-95 |
|--------------------|-----------|--------|-------|----------|
| 1-class waste item | 0.871     | 0.620  | 0.700 | 0.492    |
| 18-class object    | 0.603     | 0.361  | 0.383 | 0.289    |

YOLO26n, 2.4 million parameters, trained on the TACO litter dataset. 4.2 ms per image on the laptop GPU.

## 8. Questions about the machine learning (answered with evidence)

### Is the model overfitting or underfitting?

First model: train 96.89%, validation 89.51%, test 89.30%. Not underfitting (high training accuracy). A gap of 7.6 points, expected for 50 million parameters and 19.7 thousand images. Not harmful: validation never turned down and validation and test differ by only 0.21 points. Learning curve: more data of the same kind gives little gain now. Validation curve: the same model underfits at a strong regularisation and overfits at a weak one, the usual bias-variance picture.

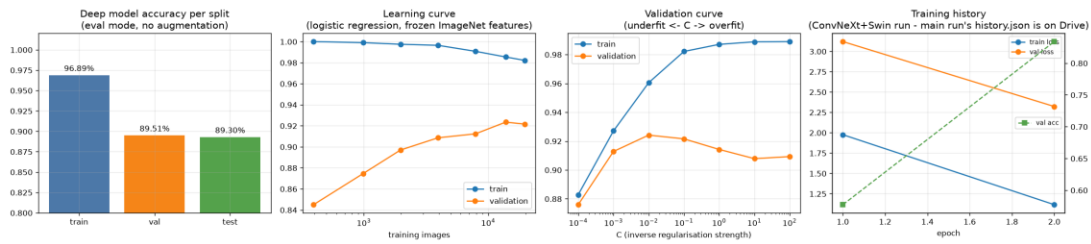


Figure 8. Fit diagnostics

## Is the confidence honest? (calibration)

The first model was under-confident: right 89% of the time but only 73% sure on average. One number, the temperature  $T = 0.71$  (fitted on validation), lowers the calibration error (ECE) from 16.6% to 7.3% and changes no prediction. Code: analysis/calibration.py.

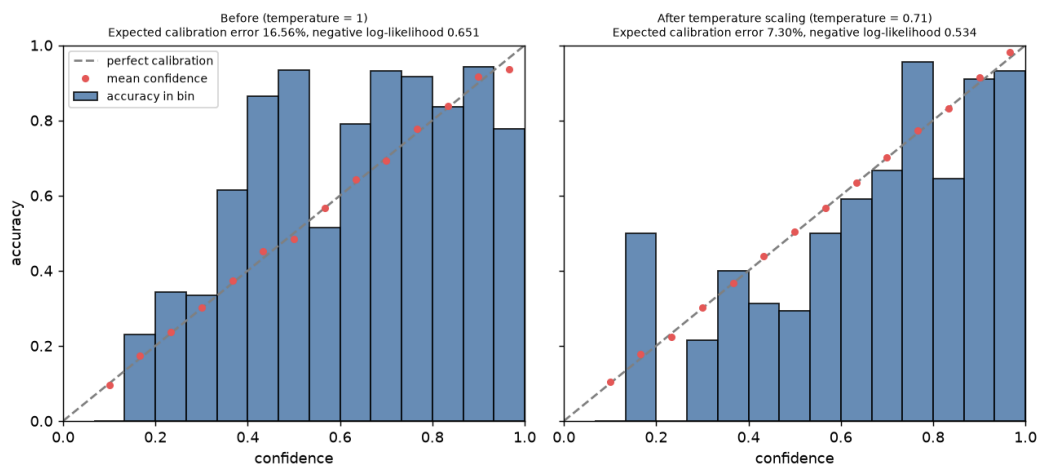


Figure 9. Reliability diagram before and after temperature scaling

## Is class imbalance handled?

Largest class 11.6 times the smallest. The first run corrected it twice (sampler and loss weights), which over-weights rare classes (about 1 over  $n$  squared). We found this and now correct once. Adjusting the scores after training ( $\text{score} + \tau * \log \text{class size}$ ,  $\tau$  chosen on validation) moved accuracy from 89.30% to 89.60% and hazard recall from 93.5% to 94.7%: a small but real effect.

## Are PCA, LDA, SVM done?

Yes. Seven classical classifiers (naive Bayes, LDA, logistic regression, linear SVM, RBF SVM, kNN, random forest) were trained after PCA on three feature sets (hand-made colour and HOG features, frozen ImageNet features, our fine-tuned features), with the same protocol as the deep model (hyperparameters on validation, test scored once). Code: analysis/classical\_baselines.py.

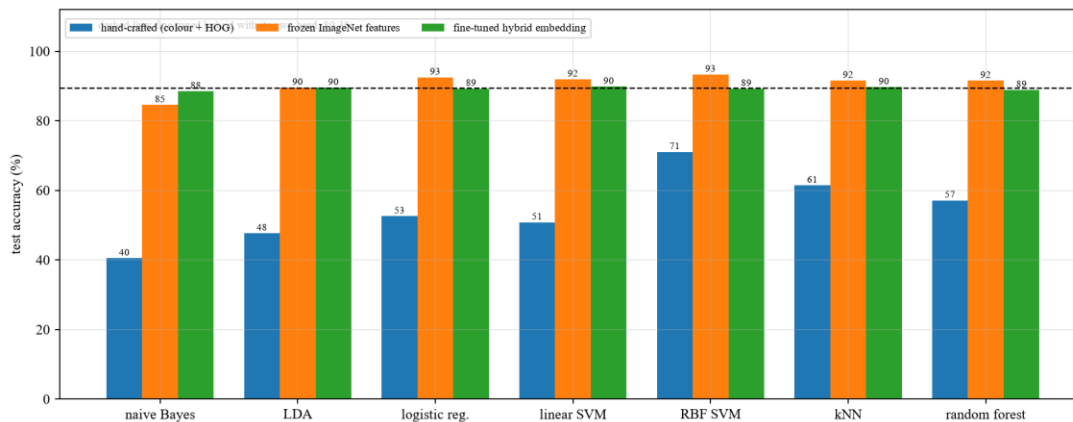


Figure 10. Test accuracy of the classical classifiers

- Hand-made features: best 71.1% (RBF SVM). Deep features: 89% to 93%. So deep learning is needed.
- Naive Bayes and LDA: small train-test gap but low accuracy (high bias). Random forest and kNN: about 99% train accuracy but big drop on test (high variance).
- RBF SVM on frozen ImageNet features: 93.3% item, 97.1% family, 98.1% hazard recall. This beat our first fine-tuned model and led to the improved recipe.
- PCA: the fine-tuned embedding keeps 95% of its information in 29 numbers (715 for hand-made features, 513 for frozen features).

Bootstrap 95% intervals for the first model: item 88.42 to 90.19%, family 93.43 to 94.80%, hazard recall 91.63 to 95.13%. We did not run k-fold cross validation for the deep model, because one run costs about an hour of GPU; the bootstrap interval on 4,232 test images already shows the uncertainty.

## 9. The safety mechanisms (the main ideas)

### 9.1 Uncertainty (monte carlo dropout)

Idea: run the classifier head 25 times with dropout switched on. If the answers agree, the model is sure.  $U$  = entropy of the average answer divided by  $\log(33)$ , so  $U$  is between 0 (sure) and 1 (unsure). The backbones have no dropout, so repeating only the head gives exactly the same result as repeating the whole network, about 100 times cheaper. Code: `decision_engine/mc_dropout_uncertainty.py`.

Honest result: as a detector of wrong answers, monte carlo dropout (AUROC 0.598) was not better than the plain highest probability (0.657). A fixed threshold of 0.5 sent 34% of items to review. So the threshold is set from a review budget (for example the 10% most unsure calibration items), not fixed.

### 9.2 Two-tier hazard gate

A confident hazard goes to the hazardous bin. An unsure hazard goes to priority review and never to recycling. On a test clip of unfamiliar items, the unsure classifier called 12 frames hazardous by chance; with the gate, automatic hazardous actions fell from 12 to 0.



Figure 11. Two-tier hazard gate

### 9.3 Conformal family-set routing (a stated limit)

Problem: a battery may be read as office paper, and then the single top answer sends it to recycling. Idea: add the class probabilities into family totals, find a threshold for each family on labelled calibration data (stricter for hazardous), and build a SET of possible families for each new item. If 'hazardous' is in the set, the item cannot go to a recycling bin. Then the chance that a hazard reaches a non-hazard bin is at most  $\alpha_H$  (the operator's choice) for items that look like the calibration items.

| $\alpha_H$ | Hazard leakage (conformal sets) | Items sent to review | Matched confidence gate |
|------------|---------------------------------|----------------------|-------------------------|
| 0.05       | 2.84%                           | 5.7%                 | 3.50%                   |
| 0.02       | 1.57%                           | 7.8%                 | 3.55%                   |
| 0.01       | 0.78%                           | 31.1%                | 1.77%                   |

Without any gate, 6.5% of hazards (48 of 738) reach recycling. At about 8% review, conformal sets leak 2.3 times less than a tuned confidence gate. Measured leakage stays below  $\alpha_H$  for every  $\alpha_H$  of 0.05 or less (300 random splits). Limit: the guarantee does not hold for very different images. With heavy noise (item accuracy 53.9%), leakage rises to 15.6% for sets alone, and an extra uncertainty gate brings it to 9.8% (top-1 alone gives 28.5%). Code: `decision_engine/conformal_routing.py`.

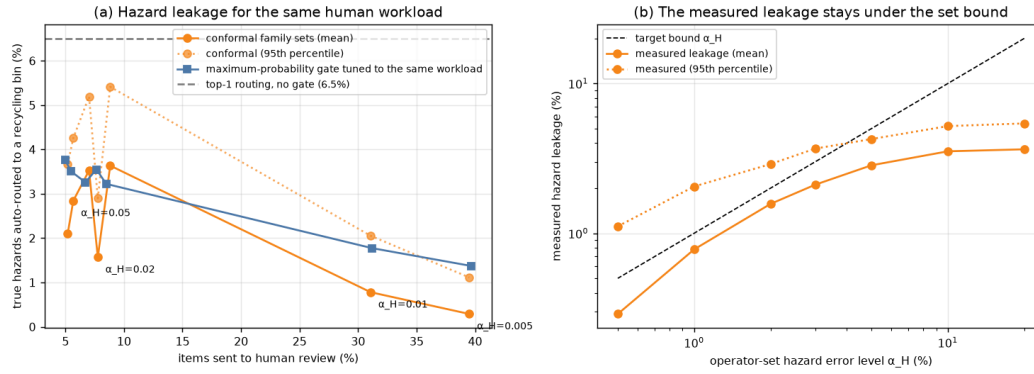


Figure 12. Leakage against review workload

## 9.4 Object-material cross-check

The detector says what the object is. A can is not plastic, so plastic gets weight 0.15 and fitting materials weight 1; then the probabilities are renormalised. Hazardous classes always keep weight 1, so a hazard call can never be hidden by this step. If the two stages still disagree, the item goes to manual review. Example from the test set: an aluminium soda can first read as a plastic bottle (69%); after the cross-check the can rises from 4% to 17% and the bottle falls to 40%. Code: `decision_engine/object_identity_prior.py`.

## 9.5 Organic contamination index (OCI)

A greasy pizza box is cardboard but cannot be recycled, and a camera cannot see grease. Moisture and gas readings are scaled to 0..1 with fixed anchors measured once (dry and wet reference; clean air and smelly reference). Three small logistic models are fitted: both sensors, moisture only, gas only. If a sensor fails, the matching model is used. The threshold is the one with the lowest false alarms among those that find at least 95% of contaminated items (missing dirt costs more than an extra check).

Result on SIMULATED sensors: with the gas channel lost, the dedicated moisture model keeps 94.9% sensitivity; putting zero for the missing gas value in the big model drops it to 53.7%. The simulated sensors are only weakly separable (false positive rate about 80% at 95% sensitivity), so real calibration is essential. Code: `contamination/oci_model.py`, `sensor_normalization.py`.

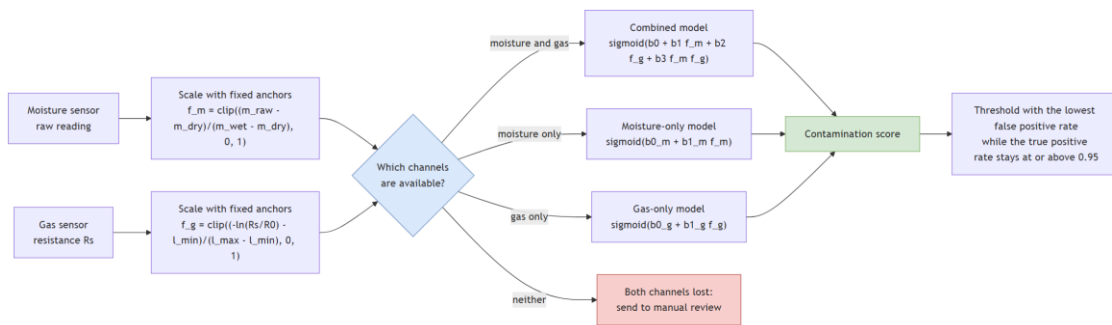


Figure 13. Organic contamination index

## 9.6 Decision engine

| Condition                                            | Action                           |
|------------------------------------------------------|----------------------------------|
| hazardous is the only family in the set and U is low | hazardous bin                    |
| hazardous is in the set in any other case            | priority review, never recycling |

|                                          |                       |
|------------------------------------------|-----------------------|
| object and material contradict           | manual review         |
| set empty or several families, or U high | manual review         |
| index above threshold on a recyclable    | contamination reject  |
| otherwise                                | bin of the one family |

Code: `decision_engine/routing_rules.py`, function `decide`. The actuator is a mock (it prints the action) until hardware exists.

## 10. Other parts

- Explainability: Grad-CAM (heat map on the ConvNeXt stream), LIME (hide parts of the image and see what changes), SHAP (share of each sensor in the contamination score; adds up exactly). Code: `explainability/` and `contamination/shap_explanation.py`.
- Segmentation: MobileSAM draws the exact outline of the item inside the detector box, so background does not reach the classifier. Code: `detection/sam_segmentation.py`.
- Synthetic images: a small GAN (DCGAN) can make extra images for rare classes. It was built and tried on textile; not used in the reported models, and image quality was not measured. Code: `augmentation/gan_augmentation.py`.
- Federated learning: units train locally and send only weights; the server averages them with weights proportional to the number of images. Simulation on saved features, 5 units with different class mixes: shared model 89.4% against 89.0% for one central model and 81.4% for units alone. Code: `federated/fedavg_simulation.py` (full model) and the review notebook 05 (features only).
- Video survey: ByteTrack gives each object an id, so one object is counted once. On a 240-frame clip made from 12 photographs, 160 detections became 16 inventory entries (the clip has no true count, so this shows merging, not counting accuracy). Code: `applications/video_litter_survey.py`.
- Edge export: both models exported to ONNX; outputs match the originals within  $9.4e-6$ . Latency on the laptop: 14 ms on the GPU, 116 ms on the CPU. The Raspberry Pi was not measured.

## 11. Hardware prototype (planned, ESP32)

A bench unit: ESP32, DHT22 (humidity, used as the moisture channel), MQ-135 gas sensor, one servo that tilts a tray (left = auto-sorted, right = hazardous, level = ask a person). The laptop runs the models and sends commands over USB serial. Parts cost roughly 1,000 to 1,800 rupees (estimate). Files: `hardware/esp32/sorter_node_esp32/sorter_node_esp32.ino`, `scripts/prototype_sorter.py`, `scripts/calibrate_prototype_oci.py`, `scripts/check_node.py`, and the guide `docs/ESP32_Prototype_Hardware_Guide.docx`. Status: the software path is tested with a simulated node and real images. The firmware is not compiled or run on hardware yet, and the sensors are not calibrated on real items.

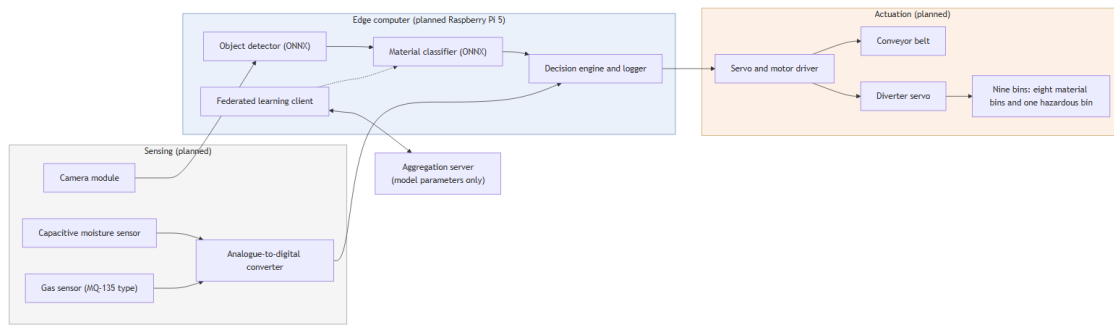


Figure 14. Planned hardware

## 12. Honest limits (what is not done)

- No physical sensors or conveyor yet: the contamination index is fitted on simulated data.
- Training photos are mostly studio or household photos. On outdoor litter the model is unsure and sends most items to a person. That is safe but shows a domain gap.
- The conformal limit is for items like the calibration items; under heavy damage it is not kept, and the uncertainty gate only reduces the risk.
- The improved model's detailed analysis (calibration, conformal) was not repeated: it was done on the first model. Its test numbers are from the Colab run.
- Monte carlo dropout is not better than the plain highest probability at finding mistakes.
- We did not test a ConvNeXt-only model, so we cannot claim that the hybrid is better than ConvNeXt alone. The attention weights only show that the model leans on ConvNeXt.
- Federated learning is a single-process simulation. Raspberry Pi latency is not measured.
- Swin Transformer was tried for only 2 epochs (83.0% item accuracy), so it is not a fair comparison.

## 13. Which feature is coded where

*This table is made from the first-line comment of every source file (each file starts with a one-line comment that says what it is).*

| file in <code>src/edgewaste/</code>               | what it does                                                                                              |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>analysis/calibration.py</code>              | checks whether predicted confidence matches real accuracy (ece, reliability diagram, temperature scaling) |
| <code>analysis/class_imbalance.py</code>          | class distribution, over-prediction check, and post-hoc logit adjustment for the imbalance correction     |
| <code>analysis/classical_baselines.py</code>      | pca, lda, svm, knn, logistic regression, random forest and naive bayes baselines on three feature sets    |
| <code>analysis/collect_predictions.py</code>      | one pass per split: classifier outputs, embeddings, frozen imagenet and hand-crafted features             |
| <code>analysis/fit_diagnostics.py</code>          | overfitting / underfitting evidence: generalisation gap, learning curve, validation curve, epoch history  |
| <code>analysis/metrics_report.py</code>           | full classification metric suite with bootstrap confidence intervals and mcnemar model comparison         |
| <code>analysis/model_complexity.py</code>         | parameter counts per component, flops, inference latency and file sizes                                   |
| <code>applications/live_camera_pipeline.py</code> | live detect-then-classify camera/image pipeline with smoothing, logging and                               |

|                                             |                                                                                                                           |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
|                                             | grad-cam                                                                                                                  |
| applications/prediction_logger.py           | writes one csv row per detection for later analysis                                                                       |
| applications/prediction_stabilizer.py       | smooths flickering per-frame predictions into one stable label per tracked object                                         |
| applications/prototype_node.py              | serial link to the arduino sensor and actuator node of the 2-day prototype, plus a mock node for testing without hardware |
| applications/video_litter_survey.py         | tracks every item through a survey video and outputs a de-duplicated litter inventory                                     |
| augmentation/gan_augmentation.py            | per-class dcgan that generates synthetic images for rare classes                                                          |
| classification/evaluate_classifier.py       | scores a checkpoint on the test split at item, family and hazard level                                                    |
| classification/export_onnx.py               | exports the classifier (and optionally the detector) to onnx and checks parity with pytorch                               |
| classification/hybrid_model.py              | convnext + vit (or swin) classifier with attention fusion of the two backbones                                            |
| classification/run_inference.py             | classifies image files or a live webcam feed with a trained checkpoint                                                    |
| classification/train_classifier.py          | trains the material classifier and saves the best checkpoint                                                              |
| common_utils.py                             | shared helpers: device selection, seeding, accuracy, parameter count, box padding                                         |
| config.py                                   | loads yaml config files into typed settings for data, model, training and detection                                       |
| contamination/oci_model.py                  | fits the three contamination models (both sensors, moisture only, gas only) and the threshold                             |
| contamination/sensor_normalization.py       | scales raw moisture and gas readings to 0-1 using fixed calibration anchors                                               |
| contamination/shap_explanation.py           | shap explanation of each contamination score: how much each sensor pushed it                                              |
| contamination/simulated_sensors.py          | simulated moisture, gas, metal and weight sensors standing in for unbuilt hardware                                        |
| contamination/synthetic_calibration_data.py | synthetic sensor calibration data, used until real sensors exist                                                          |
| data/download_kaggle_datasets.py            | downloads the public waste image datasets from kaggle                                                                     |
| data/ingest_raw_datasets.py                 | copies downloaded datasets into one folder per class, skipping unreadable images                                          |
| data/make_data_splits.py                    | writes the seeded train/val/test split manifest (data/splits.csv)                                                         |
| data/waste_image_dataset.py                 | pytorch dataset over the split manifest, with augmentation and class weights                                              |
| decision_engine/conformal_routing.py        | calibrated family prediction sets that cap how often hazards can leak into recycling                                      |
| decision_engine/mc_dropout_uncertainty.py   | prediction uncertainty from repeated dropout passes (mc-dropout)                                                          |
| decision_engine/object_identity_prior.py    | re-weights material predictions by the detector's object label, never suppressing a hazard                                |
| decision_engine/routing_rules.py            | final routing: family bin, hazardous stream, contamination reject or manual review                                        |
| detection/coco_object_recognizer.py         | coco-pretrained yolo names everyday objects and turns that into a material prior                                          |
| detection/prepare_taco_dataset.py           | downloads the taco litter dataset and converts it to yolo format                                                          |
| detection/sam_segmentation.py               | cuts an item out of its background with box-prompted mobile sam                                                           |
| detection/train_detector.py                 | trains the yolo26 waste item detector                                                                                     |
| explainability/gradcam_explanation.py       | grad-cam heatmaps of the image regions that drove the classifier's decision                                               |
| explainability/lime_explanation.py          | lime superpixel explanations for the classifier                                                                           |
| federated/fedavg_simulation.py              | fedavg simulation: clients train locally, only model weights are averaged                                                 |
| taxonomy.py                                 | the 33 waste item classes, their 9 material families and which are hazardous                                              |

## Scripts and notebooks

| script                                       | what it does                                                                                                            |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| scripts/calibrate_prototype_oci.py           | collects real sensor readings for the prototype and fits the three contamination models (the mini calibration protocol) |
| scripts/check_node.py                        | bring-up test for the esp32 or arduino node: ping, live sensor readings, tray angles and every action                   |
| scripts/collect_analysis_features.py         | caches classifier outputs and features for every split, verifying the test accuracy first                               |
| scripts/demo_contamination_index.py          | fits and checks the contamination index models on synthetic calibration data                                            |
| scripts/demo_explainability.py               | grad-cam, lime and shap explanations for one image and one sensor reading                                               |
| scripts/evaluate_conformal_routing.py        | measures hazard leakage of conformal routing against top-1 and confidence-gate baselines                                |
| scripts/finetune_detector_from_checkpoint.py | continues detector training from a saved checkpoint (runs immediately, no arguments)                                    |
| scripts/lowercase_comments.py                | turns every code comment (the text after a hash sign) in python files into lowercase, leaving code and docstrings alone |
| scripts/make_demo_gallery.py                 | builds the wider set of demonstration figures, animations and short videos for the invention disclosure draft           |
| scripts/make_demo_snapshots.py               | builds the demonstration figures and worked-example numbers used in the invention disclosure draft                      |
| scripts/prototype_sorter.py                  | runs the 2-day hardware prototype: camera + sensors + tilt tray, using the project's own classifier and decision logic  |
| scripts/reconstruct_colab_split.py           | rebuilds the exact train/val/test split the 89.30% colab model was trained and tested on                                |
| scripts/resume_detector_training.py          | resumes an interrupted single-class detector run (runs immediately, no arguments)                                       |
| scripts/run_live_camera.py                   | one-command launcher for the live camera demo with the main 33-class classifier                                         |
| scripts/run_live_camera_professor_dataset.py | launcher for the live camera demo with the professor-dataset classifier                                                 |
| scripts/run_ml_analysis.py                   | runs the full ml analysis suite on cached features and writes reports/ml_analysis                                       |
| scripts/run_simulations.py                   | simulations that test each claimed mechanism on real held-out data (vision) or synthetic sensors (labelled)             |
| scripts/train_detector_multiclass.py         | trains the 18-class taco litter detector (runs immediately, no arguments)                                               |
| scripts/train_gan_augmentation.py            | trains a dcgan for one minority class and writes synthetic images                                                       |

| notebook (review/notebooks/)         | what it shows                                                                       |
|--------------------------------------|-------------------------------------------------------------------------------------|
| 01_data_and_taxonomy                 | classes, split, imbalance, augmentation                                             |
| 02_model_code_and_training           | model code, parameter counts, training settings, training curves                    |
| 03_results_and_analysis              | accuracy, confusion, calibration, imbalance, classical baselines, pca               |
| 04_safety_mechanisms                 | uncertainty, conformal routing, cross-check, contamination index, decision engine   |
| 05_live_demo_federated_video         | full pipeline on real images, grad-cam, federated simulation, video inventory       |
| 06_colab_training_executed_annotated | the real Colab training run with explanations (where each feature is coded and why) |

## 14. Ten-minute demo plan for the review

| Minute | Show                                                                                                 | Say                                                          |
|--------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 0-1    | slides 1-5 or this dossier section 1-2                                                               | problem and goals in one minute                              |
| 1-3    | notebook 06 (Colab run): scroll the steps, show the training log and the final 93.83 / 97.28 / 98.37 | this is the real run; point to the where-is-it-coded table   |
| 3-5    | notebook 02: model code, parameter count, attention weights                                          | two networks, attention decides how much to trust each       |
| 5-7    | notebook 04: conformal table, cross-check example, contamination index                               | safety: stated hazard limit, unsure goes to a person         |
| 7-9    | notebook 05: eight real images with decisions, grad-cam, video inventory                             | the working system                                           |
| 9-10   | limits and next step                                                                                 | hardware prototype with ESP32; sensors are simulated for now |

*Commands: all notebooks open from review/notebooks and run from the project folder (they find it by themselves). The live pipeline: `python scripts/prototype_sorter.py --mock-node --image <photo>`.*

## 15. Questions the professor may ask

### **Q: Why 33 classes?**

A: Faculty asked for more than 15 to 20 classes. 33 fine classes in 9 families also let us report recognition and routing separately.

### **Q: Which optimiser, activation, loss?**

A: AdamW (lr  $3e-4$ , weight decay 0.05), GELU in all hidden layers and softmax for outputs, cross-entropy with label smoothing 0.1.

### **Q: Is the model overfitting?**

A: Mildly, not harmfully: train 97.5% vs validation 93.8% in the Colab run; best epoch 8; early stopping after 13. Regularisation: pretraining, augmentation, dropout, weight decay, label smoothing, early stopping.

### **Q: How is class imbalance handled?**

A: Balanced sampler. We found the first run corrected twice (sampler and loss weights), measured the effect (small) and now correct once.

### **Q: Did you use PCA, LDA, SVM?**

A: Yes, as baselines and analysis (section 8, notebook 03). Best classical on hand-made features 71.1%; RBF SVM on frozen deep features 93.3%.

### **Q: Why a hybrid of two networks?**

A: Convolution is good at texture, attention at shape. The learned weights show ConvNeXt gets 82-93%. We did not test ConvNeXt alone, so we do not claim the hybrid is better than it.

### **Q: Why transfer learning?**

A: 28 thousand images are too few to train 50 million parameters from scratch; from scratch gave 49% in an earlier try.

**Q: How do you know the test set is clean?**

A: Stratified fixed split, choices made on validation only, two leaks found and fixed, and the evaluation command checks the validation accuracy stored in the checkpoint.

**Q: Is 93.83% verified?**

A: It is printed by the executed Colab notebook on its own test split (cell outputs are saved). A local re-score was impossible because the local split differs; it gave a too-high 96.8%, which we discarded.

**Q: What is calibration?**

A: Among predictions with 80% confidence, about 80% should be right. We measured ECE 16.6% and reduced it to 7.3% with temperature 0.71.

**Q: What is monte carlo dropout?**

A: Run the head many times with dropout on; the spread of answers is the uncertainty. It is approximate Bayesian inference.

**Q: Is monte carlo dropout good?**

A: Not better than the plain highest probability at finding mistakes (AUROC 0.60 vs 0.66). The benefit comes from the two-tier gate and a threshold set from a review budget.

**Q: What is conformal prediction?**

A: A way to output a set of possible answers that contains the truth with a chosen probability, for new data like the calibration data. We use one threshold per family and a stricter one for hazards.

**Q: What does the guarantee cover?**

A: Items that look like the calibration items. Not very different images; there the uncertainty gate helps but the limit is not kept.

**Q: Why not accuracy only?**

A: Accuracy hides the cost of a missed battery. We report family accuracy (the bin) and hazard recall separately.

**Q: What is the contamination index?**

A: A logistic model on two sensor readings that tells if an item is dirty with food. Three models so that one sensor failing does not break it.

**Q: Are the sensors real?**

A: Not yet. Results use simulated sensor data. The ESP32 prototype is planned and its code is ready but not run on hardware.

**Q: Is federated learning real?**

A: A simulation with five units on saved features; only weights are exchanged. Not tested on physical devices.

**Q: Edge deployment?**

A: Models are exported to ONNX (match within 9.4e-6). 14 ms on the laptop GPU and 116 ms on the CPU; Raspberry Pi not measured.

**Q: How is it explained?**

A: Grad-CAM, LIME, SHAP and the SAM outline. Grad-CAM covers only the ConvNeXt stream.

**Q: What is new in your work?**

A: Mostly the combination: hazard-exempt cross-check, two-tier hazard gate, conformal routing by material family with a stricter hazard level, and a contamination index that survives a sensor failure. The single techniques (conformal prediction, dropout uncertainty, FedAvg) are published and are not claimed alone.

**Q: What are the limits?**

A: See section 12: no real sensors, domain gap, conformal limit under shift, no ConvNeXt-only test, simulated federated learning.

**Q: What is Technology Readiness Level 3?**

A: Proof of concept: the idea works in software on real data. Hardware validation is the next level.

**Q: Where is X coded?**

A: Section 13 gives a file for every feature; notebook 06 gives the training pipeline step by step.

**Q: What did you do about the detector?**

A: YOLO26n was fine-tuned on TACO for a 1-class locator (mAP50 0.700) and an 18-class object namer (0.383). The object name feeds the cross-check.

**Q: What happens when the system is unsure about a hazard?**

A: It goes to priority review, kept out of every recycling bin, never to an automatic gate.

## 16. Words explained

| Word                 | Simple meaning                                                               |
|----------------------|------------------------------------------------------------------------------|
| item / family        | item = exact class (plastic water bottle); family = the bin (plastic)        |
| hazard recall        | of all dangerous items, the share that were called dangerous                 |
| transfer learning    | start from a network that already learned on millions of pictures            |
| fine-tuning          | keep training that network on our data                                       |
| attention            | a way to give more weight to the more useful part of the input               |
| softmax              | turns scores into probabilities that add up to 1                             |
| label smoothing      | do not ask for 100% on the true class; makes the model less over-confident   |
| epoch                | one pass over all training images                                            |
| early stopping       | stop training when validation stops improving                                |
| calibration / ECE    | do the confidence numbers match the real accuracy / the size of the mismatch |
| temperature scaling  | divide scores by one number to fix over- or under-confidence                 |
| monte carlo dropout  | run the model many times with random dropout; spread = uncertainty           |
| conformal prediction | make a set of answers with a chosen chance of containing the truth           |

|                    |                                                                                 |
|--------------------|---------------------------------------------------------------------------------|
| leakage            | a hazard sent to a recycling bin (also: test images that were used in training) |
| review workload    | share of items sent to a person                                                 |
| PCA                | reduce many features to a few that keep most information                        |
| LDA                | a linear method that finds directions that separate classes                     |
| SVM                | a classifier that finds the widest margin between classes                       |
| bias / variance    | too simple a model (bias) vs too sensitive to the training set (variance)       |
| federated learning | learn together by sharing model weights, not data                               |
| ONNX               | a file format to run a model on other devices                                   |
| TRL                | scale 1 to 9 of how ready a technology is                                       |